4.5 Core data types

Elasticsearch provides a set of core data types to represent data, including `text`, `keyword`, `date`, `long`, and `boolean`. We need a basic understanding of core data types before we can understand the advanced types and dynamic and explicit mapping. In the next few sections, we look at the core basic data types with examples.

> **NOTE** To avoid bloating the chapter, the code for some of the examples can be found on GitHub (http://mng.bz/GyBR) and the book's website (https://www.manning.com/books/elasticsearch-in-action-second-edition).

4.5.1 The text data type

If there's one type of data that search engines must do well, it is the *full-text* data type. Human-readable text, also called *full text* or *unstructured text* in search engine lingo, is the bread and butter of a modern search engine. We consume a lot of textual information in the current digital world: news items, tweets, blog posts, research articles, and much more. Elasticsearch defines a dedicated data type to handle full-text data: the `text` data type.

Any field stamped with the `text` data type is analyzed before it is persisted. During the analysis process, analyzers massage (enrich, enhance, and transform) textual data into various forms and store them in internal data structures for easy access.

As we saw in our earlier examples, setting the type is straightforward. Use this syntax in your mapping definition: `"field name":{ "type": "text" }`. We worked through an example in section 4.2.2, so we won't repeat that exercise. Instead, let's look at how Elasticsearch treats `text` fields during indexing.

ANALYZING TEXT FIELDS

Elasticsearch supports both types of text data—structured and unstructured. Unstructured text is full-text data, usually written in a human-readable language (English, Chinese, Portuguese, etc.). Efficient and effective search on unstructured text makes a search engine stand out.

Unstructured text undergoes an analysis process whereby data is split into tokens, characters are filtered out, words are reduced to their root word (stemmed), synonyms are added, and other natural language processing rules are applied. We dedicate chapter 7 to text analysis, but let's quickly see an example of how Elasticsearch handles full text. The following text is a user's review of a movie:

`"The movie was sick!!! Hilarious :) :) and WITTY ;) a KiLLer` 👍`"`

When this document is indexed, it undergoes an analysis based on the analyzer. Analyzers are text-analysis modules that analyze incoming text to tokenize and normalize it. By default, Elasticsearch uses a *standard analyzer,* and analyzing the review comment involves the following steps (figure 4.7):

1. Tags, punctuation, and special characters are stripped away using character filters. This is how the review looks after this step:

   `The movie was sick Hilarious and WITTY a KiLLer` 👍